
A BUILDER'S AUDIT MANUAL

Evaluating Coding Systems

A structured audit for vibecoded & human-written code — turning “it feels right” into “I have verified it.”

Contents

0 · Clarifying Questions	2
1 · Introduction & Philosophy	5
2 · The Evaluation Checklist	7
Phase 1 — Code Comprehension & Logic Audit	7
Phase 2 — Debugging Readiness & Defect Detection	8
Phase 3 — Error Handling & Resilience	9
Phase 4 — Security Audit	10
Phase 5 — Architecture & Modularity	11
Phase 6 — Testing & Verification	13
Phase 7 — Scaling & Performance	14
Phase 8 — Production Viability	15
Phase 9 — Necessity & Redundancy	16
Phase 10 — Maintenance & Long-Term Health	17
Phase 11 — Debugging the AI Tools	19
3 · The Vibecoded Code Risk Matrix	21
4 · The Rapid Audit	24
5 · Decision Framework: Ship, Fix, or Rebuild	26
6 · Appendices	28
A — Glossary	28
B — Static Analysis & Security Tools	29
C — Vibecode Audit Report Template	29
D — Questions to Ask Your AI	31

Print this manual or keep it open beside your editor. Each phase is self-contained — jump to any phase you need. The checkbox squares are meant to be ticked.

FRONT MATTER

Section 0 · Clarifying Questions

Before you audit anything, fix your context. The right audit for a throwaway prototype is the wrong audit for a healthcare API. Answer these twelve questions first — they set your *track* and tell you

which phases to weight most heavily. Write your answers down; an honest answer here saves a misdirected audit later.

- 1 What type of system did you vibecode?** (web app, API, CLI tool, data pipeline, ML model, embedded system, game, desktop app, other)
Why it matters: the dominant failure modes differ. A web app lives or dies on input handling and auth; a data pipeline on correctness and idempotency; an ML model on data leakage and reproducibility. Your type decides which phases carry the most weight.
- 2 What is the intended deployment environment?** (local, staging, production, cloud, on-prem, edge, serverless)
Why it matters: environment assumptions are where AI-generated code silently breaks. Hardcoded localhost, ephemeral disk on serverless, cold starts, and missing config all surface only in the target environment — not on your laptop.
- 3 What is the expected scale?** (single user, small team, enterprise, public-facing, millions of users)
Why it matters: scale converts harmless patterns into outages. An $O(n^2)$ loop and an $N+1$ query are invisible at 10 users and fatal at 100,000. Scale tells you how hard to push Phase 7.
- 4 What is the tech stack and primary language(s)?**
Why it matters: each ecosystem has its own footguns and its own tooling. The static analysers, the idiomatic structure, and the most common AI hallucinations are all language-specific (Appendix B).
- 5 What is your own coding proficiency level?** (beginner, intermediate, advanced, expert — be honest)
Why it matters: this is the single most important answer. Your proficiency is the ceiling on what you can verify. If you cannot read the code, you cannot audit it — you can only trust it. Lower proficiency means you must lean harder on tools and on Appendix D.
- 6 How was the code generated?** (fully vibecoded, vibecoded then edited, vibecoded then iteratively refined, human-written, hybrid)
Why it matters: fully vibecoded code with no human edits carries the highest hidden-defect risk and demands the full Phase 11. Hybrid code has seams where AI and human assumptions collide — audit the seams.
- 7 What is the criticality of the system?** (prototype, internal tool, client-facing, safety-critical, financial, healthcare)
Why it matters: criticality sets the cost of a defect. For a prototype, a bug is an inconvenience; for a financial or safety-critical system, it is a liability. Criticality decides whether “good enough” is allowed at all.
- 8 What is the expected lifespan of this codebase?** (throwaway, short-term, long-term, permanent)
Why it matters: lifespan governs how much Phase 10 matters. Throwaway code can be illegible if it works once. Code that lives for years must be maintainable by a human who never spoke to your AI.
- 9 Are there compliance or regulatory requirements?** (GDPR, HIPAA, PCI-DSS, SOC 2, none, other)
Why it matters: compliance turns Phase 4 from advisable to mandatory and adds non-negotiable rules (data residency, audit logging, encryption, retention) that AI rarely applies unprompted.
- 10 How large is the team that will maintain this code?**
Why it matters: a solo maintainer can hold context in their head; a larger team needs conventions, documentation, and a low bus factor. Team size sets the bar for Phases 5 and 10.

11 What are the known unknowns? (areas where you suspect gaps but aren't sure)

Why it matters: your own unease is signal. The section you are “afraid to touch” is almost always where the audit pays off most. Name those areas now so you don't quietly skip them.

12 What AI tools were used, and what prompts were given? (if applicable)

Why it matters: the prompt history is your map to AI-specific risk. Vague prompts produce plausible-but-wrong logic; prompts spanning many turns risk context-window loss (the AI forgetting earlier constraints). Knowing what you asked tells you what to verify in Phase 11.

FOUNDATIONS

Section 1 · Introduction & Philosophy

This manual exists to close one specific gap: the distance between *feeling* that code is correct and *knowing* that it is. Everything that follows is built to make that gap visible and then close it.

What “vibecoding” means here

Vibecoding is the practice of generating, steering, and assembling code through natural-language interaction with AI tools — describing what you want, accepting or rejecting what comes back, and iterating, without necessarily writing the code by hand. It is genuinely powerful: it collapses the distance between idea and working artifact, lets non-specialists build real things, and frees experts from boilerplate. None of this manual is an argument against it. The argument is that the same speed that makes vibecoding powerful also hides where it goes wrong.

The vibe-confidence trap

When you write code yourself, your confidence is earned line by line — you know it works because you reasoned it into existence. When code is generated for you, that link is severed. The output *looks* fluent, idiomatic, and complete, and fluency is persuasive. So you come away feeling the code is correct without having verified that it is. This is the **vibe-confidence trap**: confidence sourced from how the code reads rather than from what it does. The trap is dangerous precisely because the code usually *is* mostly right — which trains you to skip the check on the day it is subtly, expensively wrong.

The core philosophy: evaluative coding

The remedy is **evaluative coding**: engaging with generated code through structured skepticism, gap analysis, and systematic questioning rather than passive acceptance. Evaluative coding treats every generated artifact as a claim to be tested, not a gift to be received. It asks not “does this look like a solution?” but “what would have to be true for this to be a solution, and is it?” The questions in this manual are deliberately direct and second-person because the most important auditor of your code is you, and the easiest person to deceive is also you.

This manual works for human code too

Most of what follows applies to any code, however it was written — comprehension, error handling, security, testing, and maintainability are universal. But the emphasis falls on vibecoded systems because they carry a distinct risk profile that human-written code does not: hallucinated APIs that don't exist, plausible-but-wrong logic that compiles cleanly, missing edge cases the demo never hit, architecture that shifts pattern from file to file, and security practices applied by pattern-match without understanding. Each phase therefore carries a dedicated *Vibecoding-Specific Risks* section. If your code is human-written, read those sections as a checklist of mistakes you are now less likely to make — but do not assume you are immune.

Evaluative practices, not just features

There is a difference between practices that *produce* features and practices that *reveal gaps*. Writing a new endpoint produces a feature; writing a test that fails when the endpoint misbehaves reveals a gap. This manual is built almost entirely from the second kind — debugging, error handling, security checking, scaling assessment, modularity management, testing, production-viability assessment, necessity analysis, redundancy detection, and maintenance planning. These are **evaluative practices**: their purpose is not to add to the system but to find out the truth about it. A builder who only practices the first kind ships features and accumulates unknown risk; a builder who practices both ships features and *knows what they shipped*.

Evaluation goes beyond the source code

Reading the code is necessary but not sufficient. A complete evaluation reaches past the source into questions the code cannot answer about itself: Is the architecture sound? Can it actually be deployed? Is the team ready to operate it? Is the business logic *correct*, not merely consistent? Will it survive six months of maintenance? And — the question most audits never ask — *should this code exist at all?* Phases 9 through 11 are dedicated to these questions because the most expensive defects are rarely a bad line of code; they are a sound implementation of something that should never have been built.

Vibe Risk Score — take it now (before the audit)

On a scale of 1 to 10, how much of your current confidence in this codebase comes from **vibe** (it looks right, it ran once, the AI sounded sure) versus **verified understanding** (you have read it, tested it, and can explain it)? **1 = fully verified. 10 = pure vibe**. Be honest; nobody is watching but you.

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Write your number here: _____ / 10 · Date: _____. You will re-score at the end of the audit. The drop between the two numbers is the value this manual delivered.

THE AUDIT

Section 2 · The Evaluation Checklist

Eleven phases, run in order. Each reveals a different class of problem, and the order matters: you cannot audit security in code you do not understand, and you cannot judge necessity until you know what the code does. Each phase is self-contained — its objective, what to inspect, the questions to ask yourself, the red flags, the bar for “good enough,” the AI-specific risks, and how to actually run the check. The clarifying questions are the point. They are direct on purpose. If a question makes you uncomfortable, that is the question doing its job.

Phase 1 — Code Comprehension & Logic Audit

Objective. Establish that you actually understand what the code does — line by line where it matters — rather than trusting that it looks right. Comprehension is the foundation every later phase stands on.

WHAT TO CHECK

- Every non-trivial function: can you state its inputs, outputs, side effects, and purpose without guessing?
- The control flow of the critical path — the sequence of calls that does the system's core job.
- Any block you accepted without reading, especially “glue” the AI inserted between the parts you asked for.
- Data transformations: where data changes shape, and whether each transformation is correct, not just present.

ASK YOURSELF

- ? Can you explain this function's logic to a junior developer in plain English, without reading the code aloud?
- ? If you had to rewrite this from scratch without AI, could you?
- ? Are there functions whose purpose you cannot actually state?
- ? Did you accept this solution because it looked right, or because you verified it?
- ? Is there any section you are quietly afraid to touch because you don't understand it?

RED FLAGS

- You find yourself saying “I think this does...” instead of “this does...”
- Whole files you have never read end to end but assume are fine.
- Clever one-liners or dense expressions you can't decompose.
- A function that “works” but you can't say *why* it works.

PASS CRITERIA

You can walk a colleague through the critical path unaided, name the responsibility of every function on it, and there is no block in the system you are afraid to modify. “I trust it” is not a pass; “I understand it” is.

VIBECODING-SPECIFIC RISKS

AI output is optimized to *read* well, which makes incomprehension feel like comprehension. Generated code often includes more than you asked for — defensive scaffolding, speculative branches, helper functions — that you never consciously evaluated. The fluency is the hazard: smooth code lowers your guard exactly when you should raise it.

TOOLS & TECHNIQUES

- **The explain-back test:** close the editor and narrate the critical path from memory. Gaps in your narration are gaps in your understanding.
- **The rewrite probe:** pick the most important function and re-implement it yourself; compare. Divergence is where you didn't really understand it.
- **Annotate as you read:** add a one-line comment to each function stating its purpose. The ones you can't annotate are your audit targets.

Phase 2 — Debugging Readiness & Defect Detection

Objective. Confirm that when the code fails — and it will — you can find out why, and that you have actually exercised it beyond the happy path the AI demonstrated.

WHAT TO CHECK

- Whether you can reproduce the most likely failure modes deliberately.
- Behaviour on edge inputs: empty, null, oversized, malformed, boundary values, wrong types.
- Error visibility: are failures surfaced, logged, and traceable — or swallowed?
- What happens when a dependency returns something unexpected (wrong shape, partial data, timeout).

ASK YOURSELF

- ? If this code breaks at 3 a.m., can you debug it?
- ? Do you know how to reproduce the most likely failure modes on demand?
- ? Have you actually run this against edge inputs, or only the happy path?
- ? Are there silent failures — swallowed exceptions, null returns, empty catch blocks?
- ? Did the AI generate try/catch blocks that hide errors instead of handling them?
- ? What happens when a dependency returns data you didn't expect?

RED FLAGS

- `catch` blocks that are empty, or that only `log` and continue as if nothing happened.
- Functions that return `null` / `None` / empty on error, leaving the caller to guess.
- No logging at the points where things are most likely to go wrong.
- You have never seen the code fail, so you don't know what failure even looks like.

PASS CRITERIA

You can intentionally trigger the top three failure modes, observe a clear signal when each occurs, and trace it back to a cause. No error path swallows information silently.

VIBECODING-SPECIFIC RISKS

AI loves to produce *reassuring* error handling — broad try/catch wrappers that make code look robust while actually discarding the diagnostic information you need. It also tends to demonstrate only the happy path, so edge-case behaviour is unspecified and untested by construction. The result is code that appears defensive but is undebuggable.

TOOLS & TECHNIQUES

- **Fault injection:** feed deliberately broken inputs and unplug dependencies; watch what happens.
- **Grep for swallowed errors:** search for empty catches and bare returns on error paths.
- **Step through with a debugger** on a failing case rather than reasoning about it abstractly.
- **Structured logging** at boundaries so a failure leaves a trail you can follow.

Phase 3 — Error Handling & Resilience

Objective. Verify that the system degrades sensibly under failure conditions and that its error handling is a deliberate strategy, not a scatter of per-function improvisation.

WHAT TO CHECK

- Behaviour when external dependencies are unavailable (database down, API unreachable, slow, or returning errors).
- Handling of malformed, empty, and adversarial input at every entry point.
- Whether error messages are actionable for an operator or merely generic.
- Consistency: is there one error-handling strategy, or a different ad-hoc approach in every function?

ASK YOURSELF

- ? What happens when the database is down?
- ? What happens when an upstream API returns a 500?
- ? What happens when input is malformed, empty, or adversarial?
- ? Are error messages actionable for whoever operates this, or are they generic?
- ? Is there a consistent error-handling strategy, or is it ad hoc per function?
- ? Did the AI generate error handling that looks comprehensive but actually does nothing — catch-alls, generic rethrows?

RED FLAGS

- A single global handler that catches everything and shows the user “Something went wrong.”
- Retries with no backoff, no cap, and no idempotency — a quiet denial-of-service against your own dependencies.
- Different error conventions in different files (exceptions here, error codes there, silent failure elsewhere).
- Failures that cascade — one downstream timeout takes the whole system down.

PASS CRITERIA

Each external dependency has a defined failure behaviour. Inputs are validated at the boundary. There is one coherent error-handling convention, and an operator reading a logged error can tell what happened and what to do.

VIBECODING-SPECIFIC RISKS

This is the home of **security and reliability theater**: error handling that is visually thorough but functionally inert — `catch (e) { throw e; }`, generic rethrows, or handlers that log a stack trace and proceed. Because the AI generates per-prompt, the overall strategy is rarely coherent; you get local handling that never adds up to a global posture.

TOOLS & TECHNIQUES

- **Dependency-failure simulation**: point the app at a dead database or a mock returning 500s and observe.
- **Chaos-style probes** (even manual ones): kill a process, drop the network, exhaust a queue.
- **Read every catch block in sequence** and ask of each: does this *handle* the error or merely *acknowledge* it?
- **Input fuzzing** at public entry points.

Phase 4 — Security Audit

Objective. Find the ways an attacker (or an accident) could abuse the system. For anything client-facing, regulated, or handling real data, this phase is non-negotiable and failures here outrank failures everywhere else.

WHAT TO CHECK

- Every point where untrusted input enters the system, and whether each is validated and sanitized.
- Secrets: are tokens, keys, passwords, or connection strings hardcoded in source or committed to history?
- Authentication and authorization on every protected route — not just the login page, every endpoint behind it.
- Dependencies for known vulnerable versions.
- Injection surfaces: SQL/NoSQL injection, XSS, SSRF, path traversal, command injection, deserialization.
- Cryptography: are algorithms and modes current and used correctly?
- Rate limiting and abuse protection on sensitive or expensive endpoints.

ASK YOURSELF

- ? Where does user input enter the system, and is every entry point sanitized?
- ? Are there hardcoded secrets, tokens, or credentials anywhere in the source or git history?
- ? Is authentication *and* authorization checked on every protected route?
- ? Are any dependencies on known-vulnerable versions?
- ? Did the AI use deprecated or insecure crypto patterns — MD5 for passwords, ECB mode, a static IV?

- ? Is there SQL injection, XSS, SSRF, or path-traversal exposure?
- ? Did the AI add security-shaped code it doesn't understand — CORS headers that actually *widen* access, or auth checks that can be bypassed?
- ? Are secrets in environment variables and a secret manager, or pasted into the code?
- ? Is there rate limiting on sensitive endpoints?

RED FLAGS

- `Access-Control-Allow-Origin: *` on authenticated endpoints; permissive CORS added to “make it work.”
- String-concatenated SQL queries; user input reflected into HTML without encoding.
- API keys or passwords visible in the repository or in client-side code.
- `md5` / `sha1` for password hashing; encryption with a hardcoded key or fixed IV.
- Authorization checked in the UI but not enforced on the server.
- Disabled TLS verification, “allow all” certificate handling, or `eval` on external input.

PASS CRITERIA

Every input is validated server-side; no secrets in source or history; every protected route enforces both authentication and authorization on the server; dependencies are free of known critical CVEs; injection surfaces are parameterized or encoded; cryptography uses current primitives correctly; sensitive endpoints are rate-limited.

VIBECODING-SPECIFIC RISKS

AI applies security by *pattern-match*, not by threat model. It will copy the shape of a secure practice without its substance — adding a CORS header that opens rather than restricts, an auth middleware that is never actually attached to the route, or a sanitizer that misses the relevant context. Training data skews toward older patterns, so insecure or deprecated crypto appears confidently. The danger is that the code *looks* secured, which suppresses the very scrutiny it needs.

TOOLS & TECHNIQUES

- **Dependency scanning:** `npm audit`, `pip-audit`, `govulncheck`, OWASP Dependency-Check, GitHub Dependabot (see Appendix B).
- **Secret scanning:** `gitleaks` or `trufflehog` across the full history, not just the working tree.
- **SAST:** Semgrep, CodeQL, Bandit (Python), or language equivalents.
- **Map the trust boundary by hand:** trace each input from entry to use, asking what an attacker controls.
- **Cross-check against the OWASP Top 10** as a coverage list.

Phase 5 — Architecture & Modularity Management

Objective. Judge whether the codebase is organized so a human can navigate, reason about, and change it safely — or whether it is a pile of working parts that resist modification.

WHAT TO CHECK

- Navigability: could a new developer find their way around in under an hour?

- Cohesion: does each module have a single, clear responsibility, or are there god-objects doing everything?
- Coupling: circular dependencies, modules that can't change without breaking several others.
- Duplication: the same logic copied across files instead of abstracted once.
- Convention: is the file/folder layout idiomatic for the framework, or idiosyncratic?
- Separation of concerns between business logic, data access, and presentation.

ASK YOURSELF

- ? Could a new developer navigate this codebase in under an hour?
- ? Are modules cohesive (single responsibility), or are they god-objects?
- ? Are there circular dependencies between modules?
- ? Is there duplicated logic across files that should be abstracted?
- ? Did the AI invent a new pattern for every problem instead of reusing existing abstractions?
- ? Is the file and folder structure conventional for this stack, or idiosyncratic?
- ? Can you swap out one module without breaking five others?
- ? Is there separation between business logic, data access, and presentation?

RED FLAGS

- A single file of many hundreds of lines that “does everything.”
- The same validation or formatting logic pasted in three places with small variations.
- Three different ways of doing the same thing in three different modules.
- Business logic embedded in controllers or UI components; SQL scattered through view code.
- Import graphs with cycles; a change in one file mysteriously breaking an unrelated one.

PASS CRITERIA

The structure is conventional and self-explanatory; modules have single responsibilities; there are no circular dependencies; shared logic is abstracted once; and concerns are separated well enough that a module can be replaced behind its interface without a ripple.

VIBECODING-SPECIFIC RISKS

AI generates locally, one prompt at a time, with limited memory of the whole. The predictable result is **inconsistent architecture**: a fresh pattern invented for each request, duplicated helpers because the model forgot the existing one, and a layout that drifts as the session grows. No single prompt is wrong, but the assembled whole has no coherent design. This is the gap between “each piece works” and “the system is well-built.”

TOOLS & TECHNIQUES

- **Dependency-graph visualizers** (e.g. `madge` for JS, `pydeps` for Python) to expose cycles and tangles.
- **Duplication detectors** like `jscpd` or built-in linter rules.
- **The newcomer test**: hand the repo to someone unfamiliar and time how long until they find the core logic.
- **Complexity metrics** (cyclomatic complexity, file length) to flag god-objects objectively.

Phase 6 — Testing & Verification

Objective. Determine whether tests actually verify behaviour — and the behaviour that matters — or merely assert that functions execute without crashing.

WHAT TO CHECK

- Whether tests exist at all, and whether they test *behaviour* or just absence of exceptions.
- Edge-case coverage versus happy-path-only coverage.
- Presence of integration tests, not only unit tests.
- A test for the single most critical business-logic path.
- Whether tests are tautological — asserting against mocks rather than real behaviour.
- Coverage of the critical modules specifically, not just the global percentage.
- Determinism: do tests depend on network, time, randomness, or shared state?

ASK YOURSELF

- ? Do tests exist, and do they test behaviour or just that functions don't crash?
- ? Do tests cover edge cases, or only the happy path the AI demonstrated?
- ? Are there integration tests, or only unit tests?
- ? Is there a test for the most critical business-logic path?
- ? Did the AI write tautological tests that exercise mocks instead of real behaviour?
- ? What is the coverage of the most critical modules *specifically*?
- ? If you delete a line of production code, does any test fail? If not, the tests are insufficient.
- ? Are tests deterministic, or do they depend on network, time, or random state?

RED FLAGS

- Tests that assert a function returns “truthy” or simply “does not throw.”
- Mocks configured to return exactly what the assertion then checks for — a test of the mock, not the code.
- High line coverage with no edge cases, error paths, or boundary conditions.
- Flaky tests that pass or fail depending on the time of day, network, or run order.
- No test touches the one path whose failure would actually hurt.

PASS CRITERIA

The critical paths have behavioural tests that fail when behaviour breaks; edge cases and error paths are covered, not just the happy path; there is at least one integration test across real boundaries; and tests are deterministic. The mutation check is decisive: change a line of real logic and a test should go red.

VIBECODING-SPECIFIC RISKS

AI-generated tests tend to mirror the implementation the AI just wrote, which means they encode the same assumptions and miss the same edge cases — they confirm the code does what it does, not what it should. They lean heavily on mocks, producing *tautological* tests that pass regardless of correctness. A green suite from AI tests can be worse than no suite, because it manufactures false confidence.

TOOLS & TECHNIQUES

- **Mutation testing** (e.g. `mutmut`, Stryker) to measure whether tests actually catch changes — the gold standard here.
- **Coverage by module**, not just the headline number; look specifically at the critical paths.
- **The delete-a-line probe**: remove or invert a line of real logic and confirm a test fails.
- **Read the assertions**: for each test, ask what real behaviour would have to break for it to fail. If the answer is “nothing,” the test is theater.

Phase 7 — Scaling & Performance Assessment

Objective. Find where the system will bend or break as load grows, before load finds it for you. Weight this phase by the scale you specified in Section 0.

WHAT TO CHECK

- Response-time behaviour as user count rises through realistic multiples (100 → 10,000 → 1,000,000).
- Algorithmic complexity on hot paths — $O(n^2)$ or worse hiding inside loops.
- Database access patterns: N+1 queries, missing indexes, unbounded result sets.
- Caching where appropriate, and whether cache invalidation is actually handled.
- Synchronous operations that should be asynchronous or batched.
- Resource leaks: unclosed connections, files, or handles; collections that grow without bound.

ASK YOURSELF

- ? What happens to response time when users go from 100 to 10,000 to 1,000,000?
- ? Are there $O(n^2)$ or worse algorithms hiding in hot paths?
- ? Are database queries N+1 or unindexed?
- ? Is there caching where appropriate, and is cache invalidation handled?
- ? Where, specifically, will this architecture bottleneck under load?
- ? Did the AI reach for a scalable pattern that is actually overkill for this scale — microservices for a prototype?
- ? Are there synchronous operations that should be asynchronous?
- ? Is there a memory-leak risk — unclosed resources, ever-growing collections?

RED FLAGS

- A loop that issues a query per iteration (the classic N+1).
- Nested loops over user-scaled collections; sorting or scanning inside a request handler.
- Loading an entire table into memory to filter in application code.
- No indexes on columns used in `WHERE` or `JOIN`.
- Caches with no eviction or no invalidation — stale data or unbounded growth.
- Over-engineering: a message bus, queue, and three services for an app with ten users.

PASS CRITERIA

Hot paths are linear or better; queries are indexed and free of N+1 patterns; expensive work is cached or done asynchronously; resources are released deterministically; and the architecture's complexity is proportional to the actual scale — neither naive nor gratuitously distributed.

VIBECODING-SPECIFIC RISKS

AI errs in both directions. It produces the straightforward implementation that hides quadratic behaviour and N+1 queries (it optimizes for “works on the example,” and the example is small). It also **over-engineers**, proposing patterns it has seen in large systems — microservices, event sourcing, elaborate caching — that are absurd overkill at your scale and add failure modes without benefit. Match the solution to the problem, not to the most impressive pattern in the training data.

TOOLS & TECHNIQUES

- **Load testing** with `k6`, `locust`, or `wrk` to see the curve, not guess it.
- **Query analysis:** `EXPLAIN` plans and slow-query logs; ORM query-count assertions to catch N+1.
- **Profilers** for CPU and memory to locate hot paths and leaks empirically.
- **Back-of-envelope math** on the critical path before trusting any framework's defaults.

Phase 8 — Production Viability Assessment

Objective. Decide whether this could actually run in production today — operated, observed, and recovered — or whether “it works on my machine” is hiding a pile of missing operational reality.

WHAT TO CHECK

- Deployability: can it ship now, or does it assume infrastructure that doesn't exist yet?
- A build/release path — CI/CD — versus manual, error-prone deployment.
- Observability: logging, metrics, and tracing sufficient to diagnose a production incident.
- A rollback strategy for when a release goes wrong.
- Environment-specific configuration that is externalized, not hardcoded.
- A health-check endpoint; readiness and liveness signals.
- Cold-start time, where relevant; graceful degradation during partial outages.
- Log levels that are appropriate — neither a firehose nor silence.

ASK YOURSELF

- ? Can this be deployed today, or does it need infrastructure that doesn't exist yet?
- ? Is there a CI/CD pipeline, or is deployment manual?
- ? Is there observability — logging, metrics, tracing — in place for production?
- ? Is there a rollback strategy?
- ? Are there environment-specific configs hardcoded that will break in production?
- ? Is there a health-check endpoint?
- ? What is the cold-start time, if that applies?
- ? Are there graceful-degradation patterns for partial outages?

? Is the logging level appropriate — not too verbose, not too sparse?

RED FLAGS

- `localhost`, fixed ports, or developer paths baked into the code.
- Secrets and config read from source rather than environment or a secret store.
- No logs at all — or `print / console.log` debugging left in as the logging strategy.
- Deployment that exists only as steps in someone's head.
- No way to tell whether the service is healthy without poking it manually.
- A failed release that can't be undone.

PASS CRITERIA

The application is configured through the environment, ships through a repeatable pipeline, exposes a health check, emits structured logs and key metrics, can be rolled back, and degrades gracefully when a dependency is down. An operator who has never met you could run it from the documentation.

VIBECODING-SPECIFIC RISKS

AI optimizes for code that runs in the conversation, where production does not exist. So it omits the unglamorous operational layer entirely — no health checks, no metrics, no rollback, config hardcoded to whatever made the demo work. Worse, it often leaves `print`-style debugging in place of real logging. The code can be functionally complete and still completely un-operable.

TOOLS & TECHNIQUES

- **A deployment dry-run** to a clean staging environment — the fastest way to find hardcoded assumptions.
- **The twelve-factor checklist** as a yardstick for config, logging, and process discipline.
- **Structured logging and a metrics endpoint** wired before launch, not after the first incident.
- **An incident rehearsal**: simulate a bad deploy and practice the rollback.

Phase 9 — Necessity & Redundancy Analysis

This phase goes beyond the source code. It questions whether the code should exist at all.

Objective. Separate the code that earns its place from the code that is merely present. The cheapest defect to fix is the feature you delete, and AI is extraordinarily good at generating things nobody asked for.

WHAT TO CHECK

- Whether every feature maps to an actual requirement for the current milestone.
- Components generated because the AI suggested them, not because a requirement demanded them.
- Over-engineering: premature optimization, speculative abstractions, configuration systems nobody uses.
- Functionality that duplicates a well-tested library or an existing service.
- Modules whose removal would break nothing — and which therefore have no reason to exist.
- Generated boilerplate and scaffolding that was never customized and serves no purpose.

ASK YOURSELF

- ? Is every feature in this codebase actually needed for the current milestone?
- ? Are there components that exist because the AI suggested them, not because a requirement demanded them?
- ? Is there over-engineering — premature optimization, speculative abstractions, unused configuration systems?
- ? Is there functionality that duplicates an existing library or service you could have used instead?
- ? Would removing this module break anything? If not, why does it exist?
- ? Is the codebase more than ~30% larger than it needs to be?
- ? Did the AI leave boilerplate or scaffolding that was never customized and does nothing?

RED FLAGS

- Abstraction layers with exactly one implementation and no second use case in sight.
- Configuration options, feature flags, or plugin systems that nothing actually toggles.
- Hand-rolled implementations of problems a standard library already solves correctly.
- Files you could delete with no effect on behaviour or tests.
- “Just in case” code justified by a future that has not been committed to.

PASS CRITERIA

Every module traces to a requirement; abstractions are justified by present need, not speculation; nothing reimplements a reliable library; and you could not remove a meaningful chunk without losing required behaviour. The codebase is as small as it can be while still doing its job.

VIBECODING-SPECIFIC RISKS

AI is generous to a fault. Ask for a feature and it volunteers three more, complete with abstractions and config you never requested, because elaboration is rewarded in its training. It also reaches for impressive structure — factories, interfaces, layers — where a function would do. The accumulated effect is a codebase far larger and more complex than the problem, every extra line of which is surface area for bugs and a maintenance cost forever.

TOOLS & TECHNIQUES

- **Dead-code detection** (`vulture` , `ts-prune` , `knip`) to surface the unreferenced.
- **Requirement traceability**: for each module, name the requirement it serves; the unnamed are candidates for deletion.
- **The deletion test**: remove a component and run the tests — if nothing breaks, ask why it's there.
- **Library audit**: for each custom utility, check whether a maintained library already does it better.

Phase 10 — Maintenance & Long-Term Health

Objective. Judge whether you (or someone else) can still maintain this code months from now — or whether the only option will be to re-generate it from scratch and hope. Weight this by the lifespan you set in Section 0.

WHAT TO CHECK

- Whether the code is legible to a human who never prompted the AI.
- Documentation that describes what the code *does*, not what the AI intended it to do.
- Leftover TODOs, FIXMEs, and placeholder comments.
- Naming: meaningful names versus AI-generic ones (`doTheThing` , `processData` , `handleStuff`).
- Onboarding material: a README, architecture notes, or CONTRIBUTING guide.
- Dependency versions: pinned and reproducible, or floating and fragile.
- Bus factor: how many people understand this, and what happens if they leave.

ASK YOURSELF

- ? Will you be able to maintain this in six months, or will you need to re-vibecode it from scratch?
- ? Is the documentation accurate, or does it describe what the AI intended rather than what the code does?
- ? Are there TODOs, FIXMEs, or placeholder comments left by the AI?
- ? Is the code readable by a human who didn't prompt the AI?
- ? Are variable and function names meaningful, or AI-generic — `doTheThing` , `processData` , `handleStuff` ?
- ? Is there a CONTRIBUTING or architecture doc?
- ? Are dependencies pinned to specific versions, or floating?
- ? What is the bus factor of this codebase?
- ? If the AI tool that generated this is discontinued tomorrow, can a human still maintain the code?

RED FLAGS

- Comments that contradict the code, or describe intent the code doesn't fulfill.
- Generic names that hide purpose; magic numbers and unexplained constants.
- No documentation, or documentation that is plausible-sounding but wrong.
- Floating dependency ranges that will silently change behaviour on the next install.
- A codebase only its original prompter could possibly understand.

PASS CRITERIA

A competent developer unfamiliar with the project can read it, run it from the docs, and make a change with confidence. Names communicate intent, comments match reality, dependencies are pinned, and the system's knowledge is not trapped in one person's head — or in a chat session you can't reopen.

VIBECODING-SPECIFIC RISKS

Two failure modes dominate here. First, **comment-code drift**: AI writes documentation describing what it *meant* to build, which diverges from what it actually built and then misleads every future reader. Second, the maintainability cliff — code that is generatable but not *maintainable*, where the implicit plan to “just ask the AI again” fails the moment the model, the tool, or your prompt context is gone. Generic naming compounds both, erasing the intent a maintainer needs.

TOOLS & TECHNIQUES

- **The fresh-eyes review:** have someone uninvolved read a module and report what they cannot follow.
- **Lock dependencies** and commit the lockfile; scan for floating ranges.
- **Doc-versus-code audit:** check each documented claim against the implementation.
- **Linters for naming and complexity** to flag the cryptic and the generic.

Phase 11 — Debugging the AI Tools Themselves

A meta-phase: how to debug the vibecoding process, not just its output.

Objective. Catch the failure modes that are specific to how the code was *generated* — hallucinated APIs, semantically wrong-but-compiling logic, dropped constraints — and build a decision protocol for when to trust the AI and when to take over.

WHAT TO CHECK

- Whether any referenced API, function, method, or library actually exists and behaves as used.
- Semantic correctness: code that compiles and runs but does the wrong thing.
- Whether the AI silently ignored part of your prompt — and whether you noticed.
- Consistency: did the same request produce different code across sessions?
- Whether the library API used matches the installed version, or a hallucinated/outdated one.
- References to files, configs, or modules that don't exist in the project.
- Whether the AI's confidence matched its actual correctness.

ASK YOURSELF

- ? Did the AI hallucinate any APIs, functions, or library methods that don't exist?
- ? Did it produce code that compiles but is semantically wrong — right syntax, wrong behaviour?
- ? Did it ignore any part of your prompt, and did you notice at the time?
- ? Did the same prompt yield different code across sessions — and which version is right?
- ? Is the AI using an outdated version of a library's API?
- ? Did it reference a file, config, or module that doesn't exist in the project?
- ? Did the AI's confidence in its output match its correctness?
- ? For each questionable piece: should you re-prompt, or hand-edit?
- ? What is your decision protocol for when to trust AI output versus rewrite it manually?

RED FLAGS

- Method or parameter names that “sound right” for the library but aren't in its documentation.
- Imports of packages that don't exist, or calls that fail only at runtime with “no such attribute.”
- A constraint you clearly stated (a version, a pattern, a limit) quietly absent from the result.
- Code referencing `config.yaml`, a helper module, or an env var that was never created.
- The AI asserting correctness with the same confident tone whether it was right or wrong.

PASS CRITERIA

Every external reference is verified against real documentation and the installed version; the code's behaviour matches intent, not just syntax; all stated constraints are present; there are no dangling references to nonexistent files or modules; and you have an explicit, written rule for when you trust generated code and when you rewrite it.

VIBECODING-SPECIFIC RISKS

This phase exists because the generation process is itself a source of defects no human author would introduce. **Hallucination** invents plausible APIs; **context-window loss** drops constraints you set earlier in a long session; **over-confident output** decouples the AI's tone from its accuracy, so confidence is no evidence of correctness; and the model may prioritize readable, idiomatic-looking code over correct code (**tone over correctness**). None of these announce themselves — they have to be hunted.

TOOLS & TECHNIQUES

- **Verify every unfamiliar API** against official docs and the actual installed version before trusting it; run the import.
- **Self-audit prompts:** feed the code back to the AI with the questions in Appendix D — useful for generating candidates, never sufficient as proof.
- **Cross-session diffing:** regenerate a critical piece and compare; divergence flags uncertainty.
- **A written trust protocol:** e.g. “verify all external APIs; rewrite anything I can't read; re-prompt only twice before taking over by hand.”

REFERENCE

Section 3 · The Vibecoded Code Risk Matrix

These risks are specific to AI-generated code — the failure modes a human author would not introduce. Use the matrix as a lookup: when something feels off, find the category, confirm the mechanism, and apply the detection method. Severity is a default for a typical production system; raise it for anything regulated, financial, or safety-critical.

SPECIFIC RISK	WHY IT HAPPENS	HOW TO DETECT	SEVERITY
HALLUCINATION			
Calls a library method that does not exist	Model predicts a plausible name from similar APIs it has seen	Run it; check against official docs and installed version	CRITICAL
Imports a package that was never published	Name resembles real packages; pattern completion over fact	Install step fails; dependency audit; verify on the registry	HIGH
Invents config keys or env vars the framework ignores	Fills gaps with what “should” exist rather than what does	Cross-check against the framework's documented options	MEDIUM
PLAUSIBLE-BUT-WRONG LOGIC			
Off-by-one or inverted boundary condition	Surface form looks correct; the model doesn't execute it	Boundary-value tests; trace the logic by hand	HIGH
Correct algorithm applied to the wrong data	Local coherence without whole-system understanding	End-to-end test against known-correct outputs	CRITICAL
Business rule subtly misimplemented (e.g. wrong rounding)	Model guesses the rule it wasn't told precisely	Verify against the actual specification, not the demo	CRITICAL
PATTERN-MATCHING WITHOUT UNDERSTANDING			
Copies a secure-looking pattern that doesn't fit the context	Reproduces shape of training examples, not their intent	Threat-model the specific use; don't trust the shape	HIGH
Applies a concurrency idiom that introduces a race	Idiom learned without its preconditions	Stress/concurrency tests; review locking by hand	HIGH
Uses a design pattern where it actively harms clarity	Pattern recognized as “good code” regardless of fit	Ask whether a simpler construct would do the same job	LOW
INCONSISTENT ARCHITECTURE			
A different pattern for the same problem in each file	Per-prompt generation with no memory of prior choices	Read across modules; look for divergent conventions	MEDIUM
Duplicated helpers because earlier ones were forgotten	Limited context window across a long session	Duplication detector (<code>jscpd</code>); search by behaviour	MEDIUM
Folder layout drifts as the project grows	No global plan; structure improvised per request	Compare against the framework's conventional layout	LOW

SPECIFIC RISK	WHY IT HAPPENS	HOW TO DETECT	SEVERITY
OVER-ENGINEERING			
Speculative abstraction with a single implementation	Elaboration is rewarded; “flexibility” volunteered unasked	Count implementations behind each interface; ask “why now?”	MEDIUM
Microservices / queues for a tiny app	Imitates large-system patterns from training data	Compare complexity to the scale set in Section 0	HIGH
Configuration system nothing actually uses	“Best practice” added reflexively	Dead-config detection; grep for unused options	LOW
UNDER-ENGINEERING			
Naive implementation hiding quadratic complexity	Optimizes for the small example, not the real load	Load test; complexity review of hot paths	HIGH
No validation because the demo input was clean	Happy-path bias in demonstrations	Fuzz the entry points; test malformed input	HIGH
Missing concurrency/transaction safety	Single-user mental model of the problem	Concurrent-access tests; review transactional boundaries	CRITICAL
SECURITY THEATER			
Auth middleware defined but never attached to routes	Produces the artifact without wiring it in	Test protected routes unauthenticated; trace registration	CRITICAL
CORS header that widens rather than restricts access	Pattern copied without understanding its effect	Review CORS config against intended origins	HIGH
Sanitizer applied in the wrong context (misses XSS)	Right tool, wrong place; encoding context ignored	SAST; test injection in each output context	HIGH
MISSING EDGE CASES			
Empty / null / single-element inputs unhandled	Demonstrated only on typical inputs	Explicit edge-case tests; boundary fuzzing	MEDIUM
Timezone, locale, or encoding assumptions	Defaults to the most common case in training data	Test across locales, timezones, and character sets	MEDIUM
Unhandled partial failure of a dependency	Assumes dependencies always succeed	Fault injection on each external call	HIGH
OUTDATED API USAGE			
Deprecated method or removed parameter	Training data skews toward older versions	Deprecation warnings; check against current docs	MEDIUM
Insecure crypto (MD5/SHA-1, ECB, static IV)	Old patterns over-represented and confidently emitted	SAST crypto rules; manual crypto review	CRITICAL
Superseded framework idiom no longer recommended	Model trained before the API changed	Compare to the framework's current best-practice guide	LOW

SPECIFIC RISK	WHY IT HAPPENS	HOW TO DETECT	SEVERITY
PHANTOM DEPENDENCIES			
References a file or module not in the project	Assumes context that was never created	Run it; resolve every import and path	HIGH
Reads an env var or config key never defined	Generated against an imagined environment	Start with a clean env; watch for missing-config errors	MEDIUM
Assumes a service or table that doesn't exist yet	Plans for infrastructure not provisioned	Integration test against the real environment	HIGH
COMMENT-CODE DRIFT			
Comment describes intended, not actual, behaviour	Documents the plan; implementation then diverged	Read each comment against the code beneath it	MEDIUM
Docstring lists parameters the function no longer takes	Signature changed; doc not updated	Compare signatures to docstrings; doc linters	LOW
README describes features that were removed or never built	Generated aspirationally alongside the code	Verify each documented claim against reality	MEDIUM
OVER-CONFIDENT OUTPUT			
Asserts correctness with identical tone whether right or wrong	Confidence is a style, not a calibrated signal	Verify independently; never treat tone as evidence	HIGH
“Production-ready” claim on unoperable code	Optimizes for a confident, helpful register	Run the full Phase 8 regardless of the claim	MEDIUM
Explains a bug as a feature when challenged	Rationalizes its own output fluently	Test the behaviour; trust results over explanations	MEDIUM
CONTEXT-WINDOW LOSS			
Drops a constraint set earlier in a long session	Earlier instructions fall out of the active context	Re-check output against your original constraints list	HIGH
Reverts a fix it made a few turns ago	No persistent memory of prior decisions	Diff against previous versions; regression tests	MEDIUM
Forgets the chosen library and introduces a second one	Loses track of established project conventions	Audit the dependency list for redundancy	MEDIUM
TONE OVER CORRECTNESS			
Prefers readable-looking code that is subtly wrong	Fluency and clarity are rewarded over verified behaviour	Behavioural tests; don't equate “clean” with “correct”	HIGH
Simplifies away a necessary edge case for elegance	Optimizes for a tidy explanation	Check that simplifications preserve required behaviour	MEDIUM
Chooses a familiar idiom over the correct-but-ugly one	Common patterns dominate the output distribution	Verify the idiom actually fits the problem	LOW

EXPRESS LANE

Section 4 · The Rapid Audit

The “I vibecoded something and need to ship it tomorrow” version. Thirty yes/no questions, under thirty minutes, grouped by phase. Tick the box only for a confident yes. This does not replace the full audit — but it will catch the failures most likely to hurt you fastest. **Any unchecked box in Security or Production is a reason to pause.**

COMPREHENSION & DEBUGGING

- ☐ Can you explain the critical path in plain English without reading it line by line?
- ☐ Is there no block of code you are afraid to modify?
- ☐ Have you run the code against at least one empty / null / malformed input?
- ☐ Are there no empty catch blocks or silent error returns?

ERROR HANDLING & RESILIENCE

- ☐ Is there defined behaviour when the database or a key API is unavailable?
- ☐ Is input validated at every public entry point?
- ☐ Do error messages tell an operator what actually went wrong?

SECURITY

- ☐ Are there zero hardcoded secrets in the source *and* git history?
- ☐ Is auth + authorization enforced on the server for every protected route?
- ☐ Are SQL queries parameterized and user output encoded (no injection/XSS)?
- ☐ Did a dependency scan come back clean of critical CVEs?
- ☐ Is password hashing and encryption using current algorithms (no MD5/ECB)?
- ☐ Is there rate limiting on sensitive or expensive endpoints?

ARCHITECTURE & TESTING

- ☐ Could a new developer find the core logic in under an hour?
- ☐ Is there no large god-object doing everything?
- ☐ Do tests exist for the single most critical path?
- ☐ Do those tests fail if you break the logic (not just check a mock)?
- ☐ Are the tests deterministic (no reliance on time, network, or random state)?

SCALING & PERFORMANCE

- ☐ Are there no N+1 queries or unindexed lookups on hot paths?
- ☐ Is there no obvious $O(n^2)$ -or-worse loop over user-scaled data?
- ☐ Are resources (connections, files, handles) released deterministically?
- ☐ Is the architecture's complexity proportional to the real scale?

PRODUCTION VIABILITY

- ☐ Is config read from the environment, not hardcoded (no `localhost` baked in)?
- ☐ Is there structured logging (not leftover `print` debugging)?
- ☐ Is there a health-check endpoint and a way to roll back a bad release?
- ☐ Can the app be deployed from documentation by someone who isn't you?

NECESSITY, MAINTENANCE & AI RISK

- ☐ Does every module trace to an actual requirement (no orphan code)?
- ☐ Are names meaningful and comments consistent with the code?
- ☐ Have you verified that every external API and import actually exists?

Thirty items. If you cleared every box in Security and Production and most of the rest, you can ship with eyes open. If not, you now know precisely what you are choosing to risk.

THE VERDICT

Section 5 · Decision Framework: Ship, Fix, or Rebuild

An audit that ends in a list of problems is only half done. Now decide. The framework gives three outcomes and the criteria for each. Run the decision steps in order, weight failures by severity (a security failure outweighs a style issue), and answer the honesty questions at the end before you commit.

SHIP

Viable for production with minor fixes.

- No phase failed on a *critical* item; Phases 4 (Security) and 8 (Production) both pass.
- Remaining issues are cosmetic or low-severity and can be fixed without architectural change.
- You understand the code well enough to support it (Phase 1 passes).
- Technical debt is acknowledged, written down, and acceptable for this milestone.

FIX

Significant but bounded gaps, repairable in a reasonable timeframe.

- One to a few phases failed, but the failures are localized — not woven through the whole codebase.
- Security or production gaps exist but are specific and addressable (add validation, wire up auth, externalize config).
- The architecture is sound enough to build on; you are patching, not rebuilding.
- You have — or can get — the skill to make the fixes correctly.

REBUILD

Fundamentally flawed; rewrite (by a human, or with sharper AI prompting and tighter review).

- Multiple phases failed on critical items, or the failures are systemic (inconsistent architecture, pervasive insecurity, untestable design).
- You cannot understand large parts of the code and cannot safely change them.
- The cost to fix meets or exceeds the cost to rebuild — patching would mean rewriting most of it anyway.
- The code works by coincidence rather than by design, and you don't trust it under change.

How to decide

- 1 **Count the failures, weighted by severity.** A critical-severity failure counts for far more than a low one. Security and correctness failures dominate.
- 2 **Check the gates.** If Phase 4 (Security) or Phase 8 (Production) failed on a critical item for a system that handles real data or users, you are not in *Ship* — regardless of everything else.
- 3 **Ask whether the failures are localized or systemic.** Localized failures point to *Fix*; failures spread across the architecture point to *Rebuild*.
- 4 **Weigh fix-cost against rebuild-cost.** If fixing means touching most of the codebase, rebuilding is cleaner and safer.
- 5 **Match the work to your skill.** If you can't make the fixes correctly, either acquire the skill, bring in someone who has it, or rebuild with tighter constraints — do not ship hope.

BEFORE YOU COMMIT, ASK YOURSELF

- ? How many phases failed — and how many on critical items?
- ? Which phases failed? (A security failure is not the same as a style issue.)
- ? Is the technical debt acceptable for the current milestone, or are you borrowing against a future you'll regret?
- ? Do you have the skill to fix the failures, or do you need someone who does?
- ? Is the cost of fixing genuinely greater than the cost of rebuilding?
- ? Are you shipping this because it's ready — or because you're tired?

Vibe Risk Score — take it again (after the audit)

Now that you have done the work, re-score from 1 to 10. **1 = fully verified understanding. 10 = pure vibe.** Compare it to the number you wrote at the start of Section 1.

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Score now: _____ / 10 · Score at start: _____ / 10. If the number didn't drop, the audit didn't land — go back to the phases you rushed. A score still above 5 means you are shipping on vibe; treat that as a finding, not a footnote.

APPENDICES

Section 6 · Appendices

Reference material to support the audit: a shared vocabulary, tooling by language, a fill-in report you can attach to the work, and a set of questions to turn your AI into its own first auditor.

Appendix A — Glossary

Vibecoding

Generating, steering, and assembling code through natural-language interaction with AI tools, without necessarily writing it by hand.

Evaluative coding

Engaging with generated code through structured skepticism, gap analysis, and systematic questioning rather than passive acceptance — the core practice of this manual.

Vibe-confidence trap

Confidence in code drawn from how fluently it reads rather than from verified understanding of what it does.

Vibe Risk Score

A 1–10 self-assessment of how much your confidence rests on vibe versus verification; taken before and after the audit.

Evaluative practices

Coding practices designed to reveal gaps rather than produce features — debugging, testing, security checking, necessity analysis, and the rest.

Hallucinated API

A function, method, parameter, or package that the AI references confidently but which does not exist.

Plausible-but-wrong logic

Code that compiles and runs but does the wrong thing — correct syntax, incorrect behaviour.

Security theater

Security-shaped code that looks protective but does nothing — middleware never attached, sanitizers in the wrong context, permissive CORS dressed as restriction.

Phantom dependency

A reference to a file, module, config key, env var, or service that does not exist in the project.

Comment-code drift

Documentation that describes what the AI intended rather than what the code actually does, diverging over time.

Context-window loss

The AI forgetting constraints or decisions from earlier in a session as they fall out of its active context.

Over-confident output

The AI asserting correctness in the same confident tone whether it is right or wrong; tone is not a calibrated signal.

Tone over correctness

The tendency to prefer readable, idiomatic-looking code over code that is verifiably correct.

God-object

A module or class that accumulates too many responsibilities, becoming a single point that everything depends on and nothing can change safely.

N+1 query

Issuing one database query per item in a loop instead of one query for the whole set — invisible at small scale, fatal at large.

Tautological test

A test that asserts against the behaviour of a mock it configured, so it passes regardless of whether the real code is correct.

Bus factor

The number of people who would have to be lost before a codebase becomes unmaintainable; lower is riskier.

Appendix B — Static Analysis & Security Scanning Tools

A starting set by ecosystem. Run a linter, a type/static analyser, a dependency vulnerability scanner, and a secret scanner at minimum. The last two are language-agnostic and belong in every project.

ECOSYSTEM	LINT / STATIC ANALYSIS	SECURITY / SAST	DEPENDENCY AUDIT
Python	Ruff, Pylint, mypy	Bandit, Semgrep	pip-audit, Safety
JavaScript / TypeScript	ESLint, tsc, Biome	Semgrep, CodeQL	npm audit, Snyk
Java / Kotlin	SpotBugs, Checkstyle, PMD	Semgrep, CodeQL, FindSecBugs	OWASP Dependency-Check
Go	go vet, staticcheck, golangci-lint	gosec, Semgrep	govulncheck, Nancy
Ruby	RuboCop, Sorbet	Brakeman, Semgrep	bundler-audit
PHP	PHPStan, Psalm	Semgrep, Psalm taint	Composer audit, Roave SecurityAdvisories
Rust	Clippy, rustc	cargo-geiger, Semgrep	cargo-audit
C / C++	clang-tidy, cppcheck	Clang Static Analyzer, Coverity	OWASP Dependency-Check
Any language	—	Semgrep (multi-lang), SonarQube	Trivy, Gype, GitHub Dependabot
Secret scanning	gitleaks, trufflehog, git-secrets — run across full history, not just the working tree		

Appendix C — Vibecode Audit Report Template

Fill this out after running the audit and attach it to the codebase or pull request. It turns the audit into a durable record and an action list. Mark each phase Pass / Fail / N/A, note the worst finding, and capture the action items.

Project:

Auditor:

Date:

System type & criticality:

Generation method:

Vibe Risk Score — before: / 10 after: / 10

PHASE	RESULT	WORST FINDING / NOTES
1 · Comprehension	Pass ☐ Fail ☐	
2 · Debugging readiness	Pass ☐ Fail ☐	
3 · Error handling	Pass ☐ Fail ☐	
4 · Security	Pass ☐ Fail ☐	
5 · Architecture	Pass ☐ Fail ☐	
6 · Testing	Pass ☐ Fail ☐	
7 · Scaling & performance	Pass ☐ Fail ☐	
8 · Production viability	Pass ☐ Fail ☐	
9 · Necessity & redundancy	Pass ☐ Fail ☐	
10 · Maintenance	Pass ☐ Fail ☐	
11 · AI-tool debugging	Pass ☐ Fail ☐	

Decision: Ship ☐ Fix ☐ Rebuild ☐

Top action items (in priority order):

- ☐
- ☐
- ☐
- ☐

Appendix D — Questions to Ask Your AI

Feed these back to the AI that generated the code to make it self-audit. Treat the answers as *leads to verify*, never as proof — an AI checking its own work shares its blind spots. Useful for surfacing candidates fast; insufficient on its own.

- “Review this code for security vulnerabilities you may have introduced. Check specifically for injection, broken auth, exposed secrets, and insecure crypto.”
- “What edge cases did you not handle? List the inputs that would break this.”
- “Are there any APIs, methods, or packages you referenced that may not exist or may be deprecated? Verify each against the current documentation.”
- “What would break if this ran at 1000× the scale — users, data, requests?”
- “Where does this code make assumptions about the environment (paths, config, services) that won't hold in production?”
- “Which parts of my original requirements did you not fully implement, simplify, or skip?”
- “Identify any error handling here that looks comprehensive but doesn't actually handle the error.”
- “Do these tests actually verify behaviour, or do they pass regardless of whether the code is correct?”
- “Is there duplicated logic or an inconsistent pattern across these files that should be unified?”
- “What did you add that I didn't ask for, and would the code work without it?”
- “Do the comments and docs accurately describe what the code does, or what you intended it to do?”
- “Rate your confidence in each part of this code from 1 to 10, and explain what you are uncertain about.”

The goal was never to distrust the machine. It was to replace “it feels right” with “I have checked.” That difference is the whole job.